

MBA IN SOFTWARE ENGINEERING

Event Storming in Software Projects

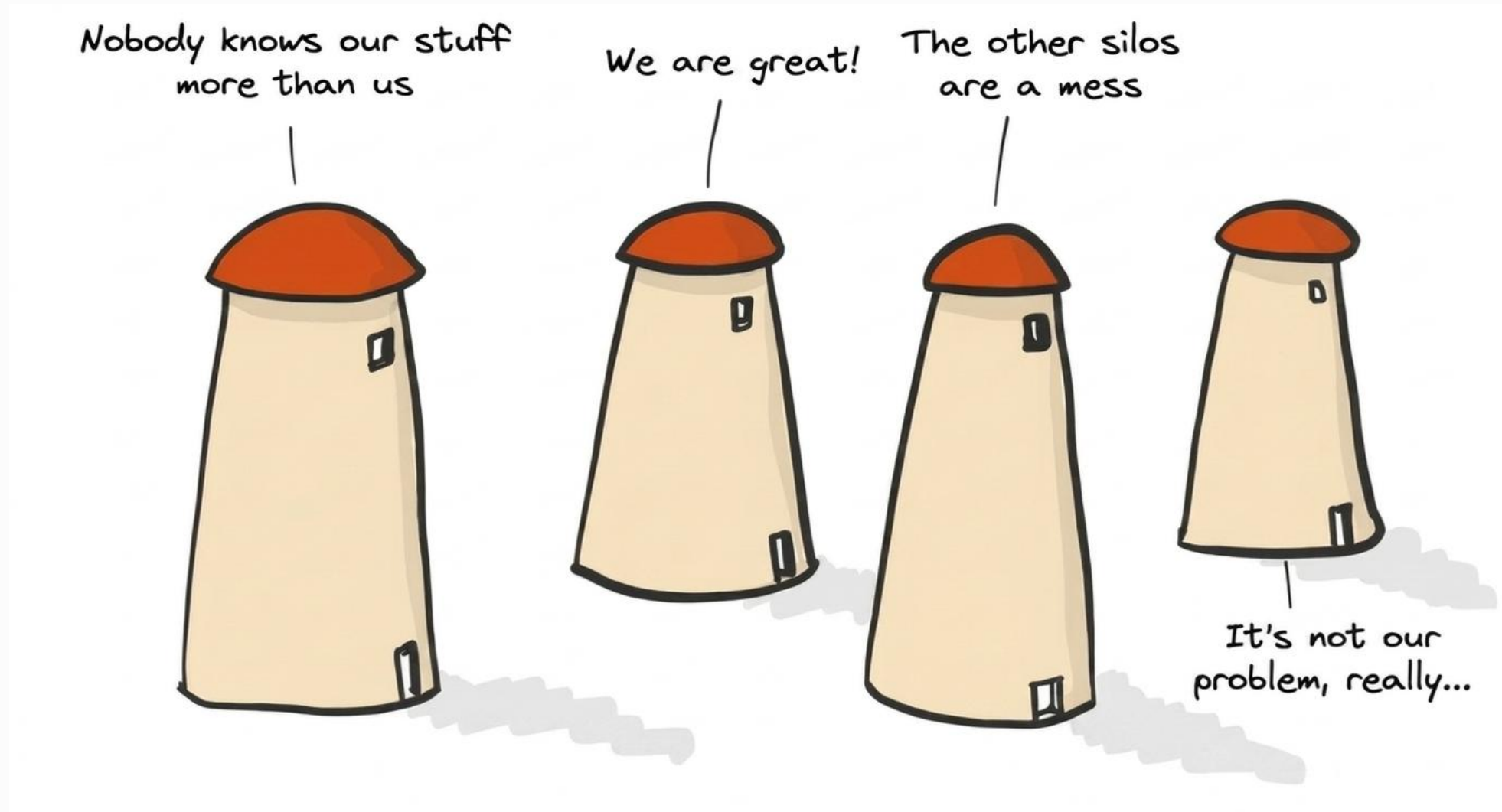
Helder Prado Santos

The trustworthiness, originality,
and legality of the educational content presented is
responsibility of the professor.

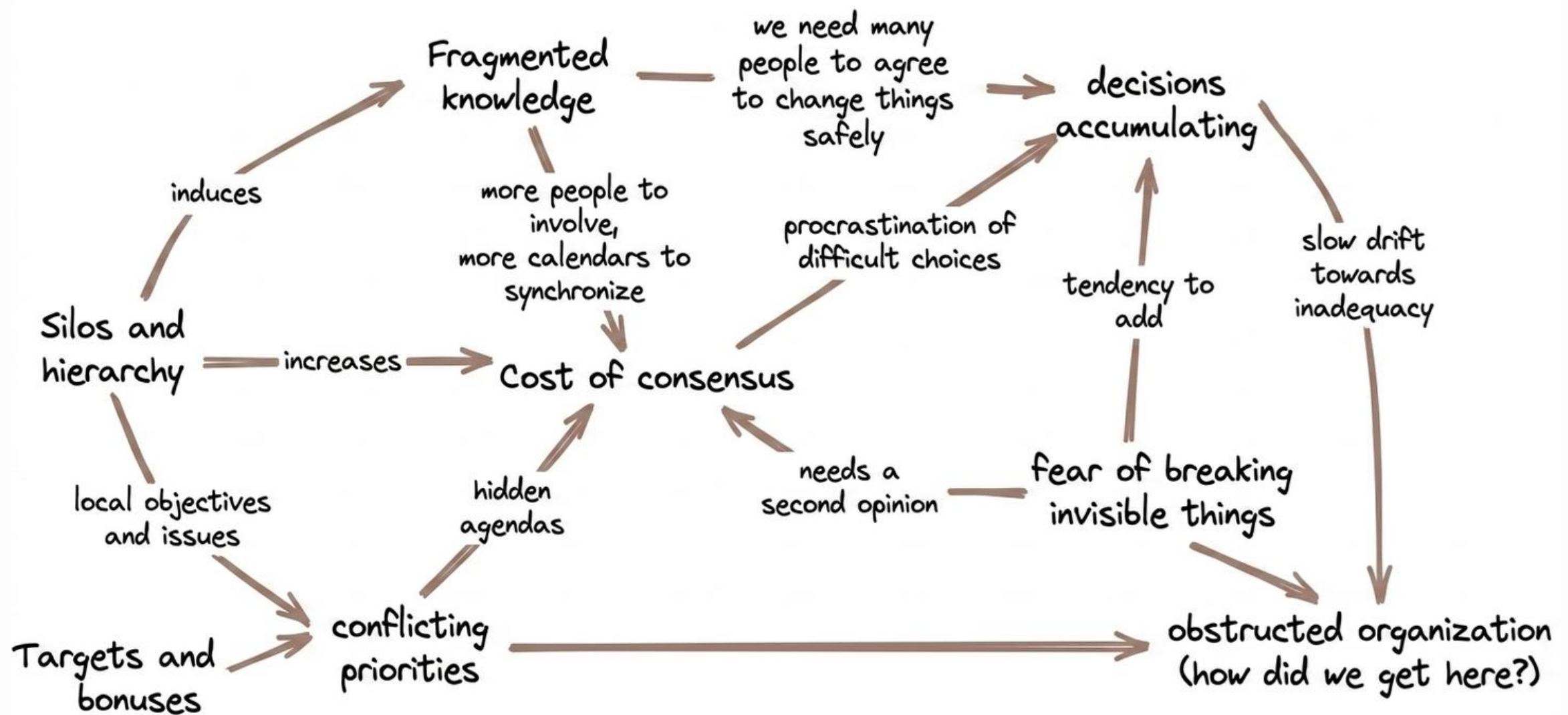
The total or partial reproduction of this material without
authorization is prohibited.

Law No.
9610/98

Organizational silos



The problem of organizational space

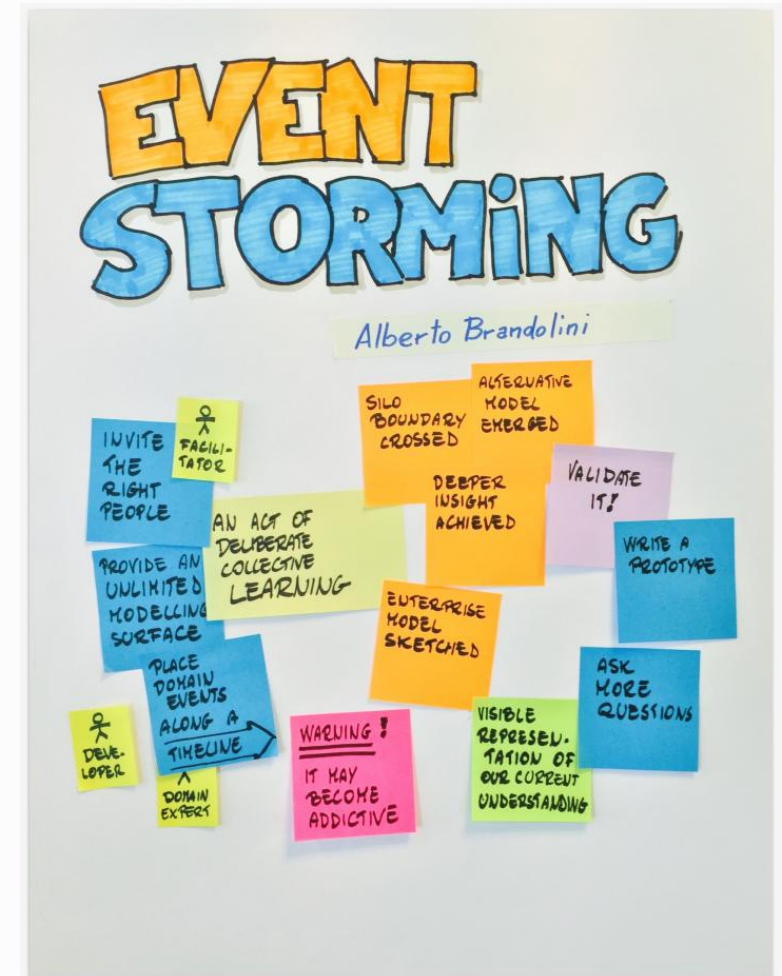


How Event Storming can help

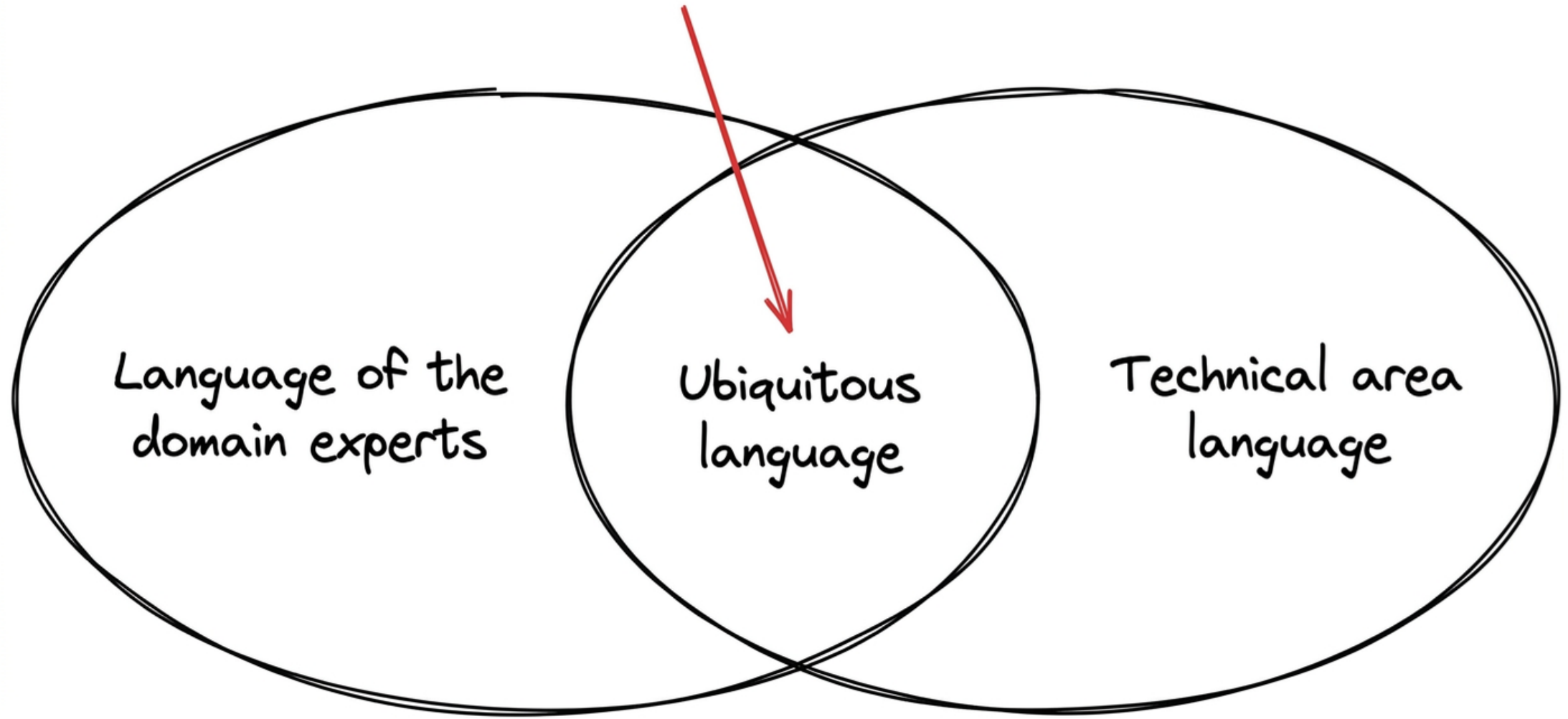


Getting to know Event Storming

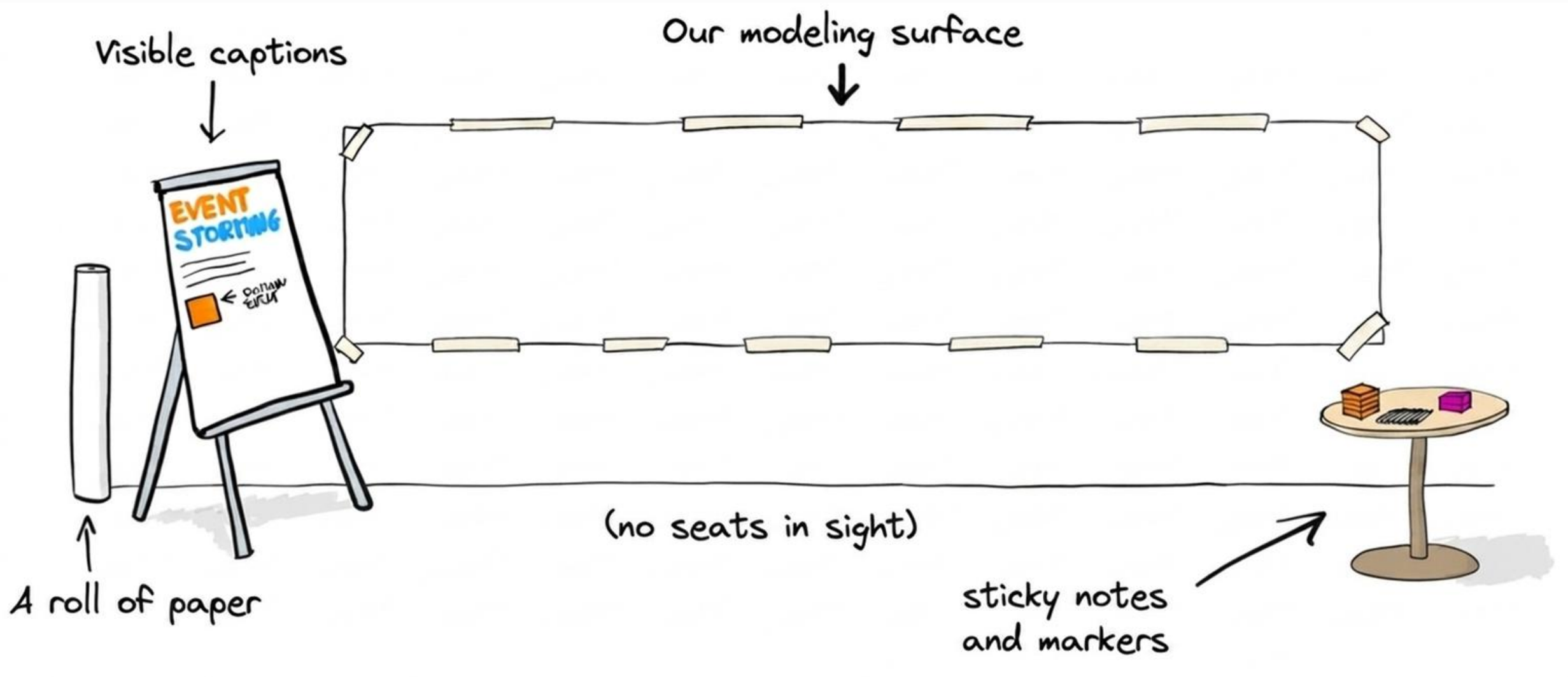
- Created by **Alberto Brandolini**
- Technique for **understanding the application domain** through its events
- Conducted in workshops involving **domain experts** and **the technical area**



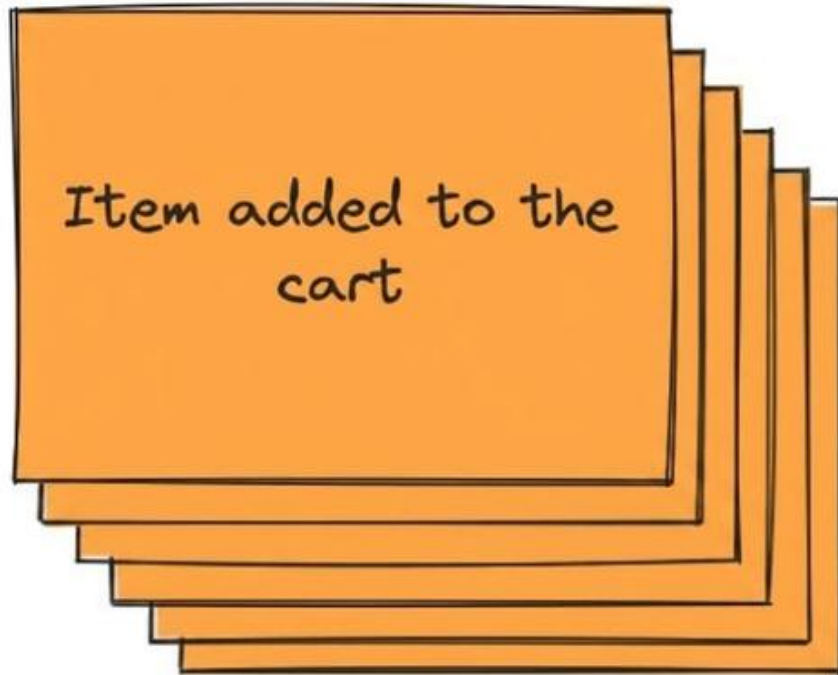
Developing the ubiquitous language



The workshop

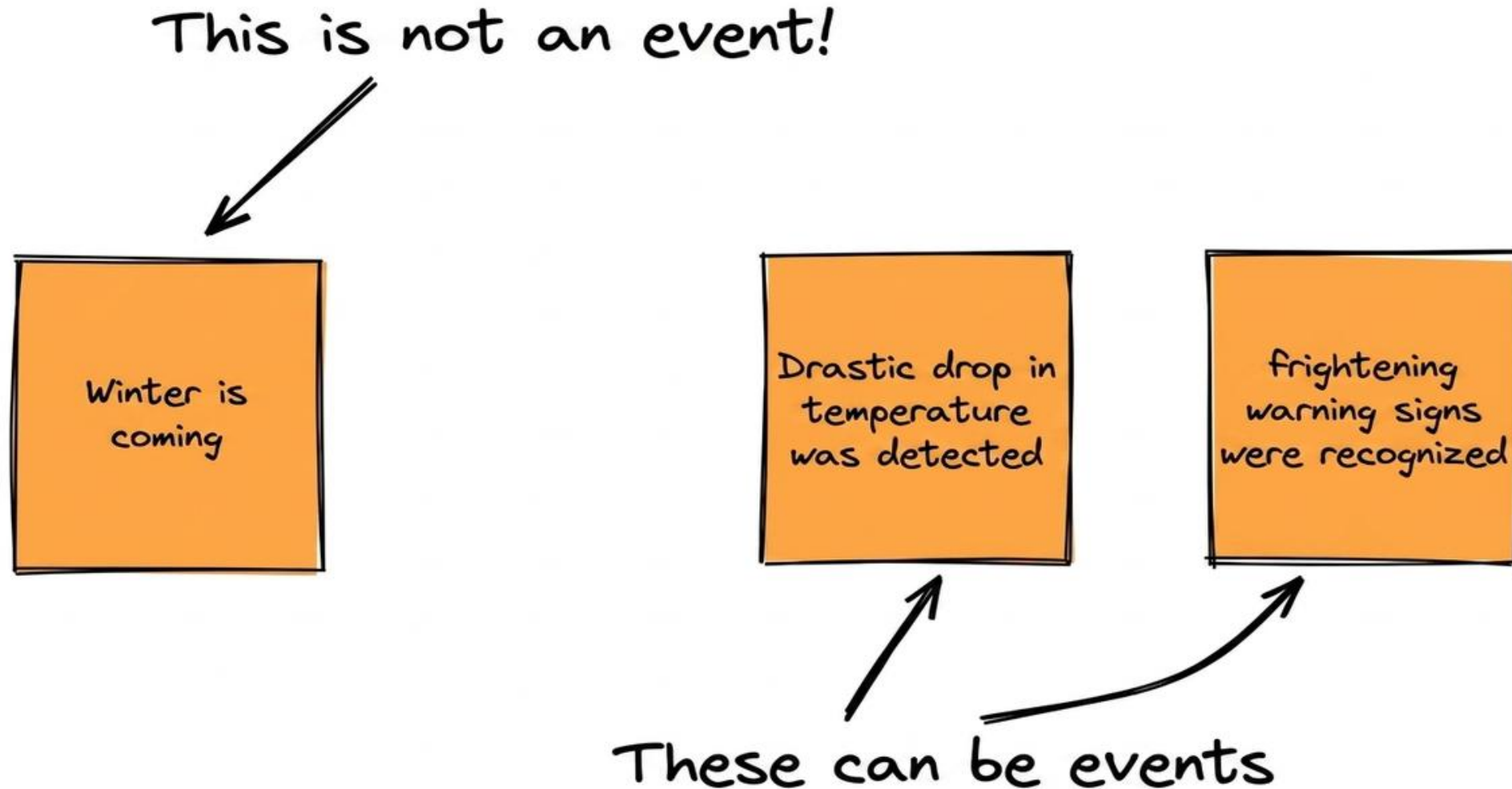


Mapping system events

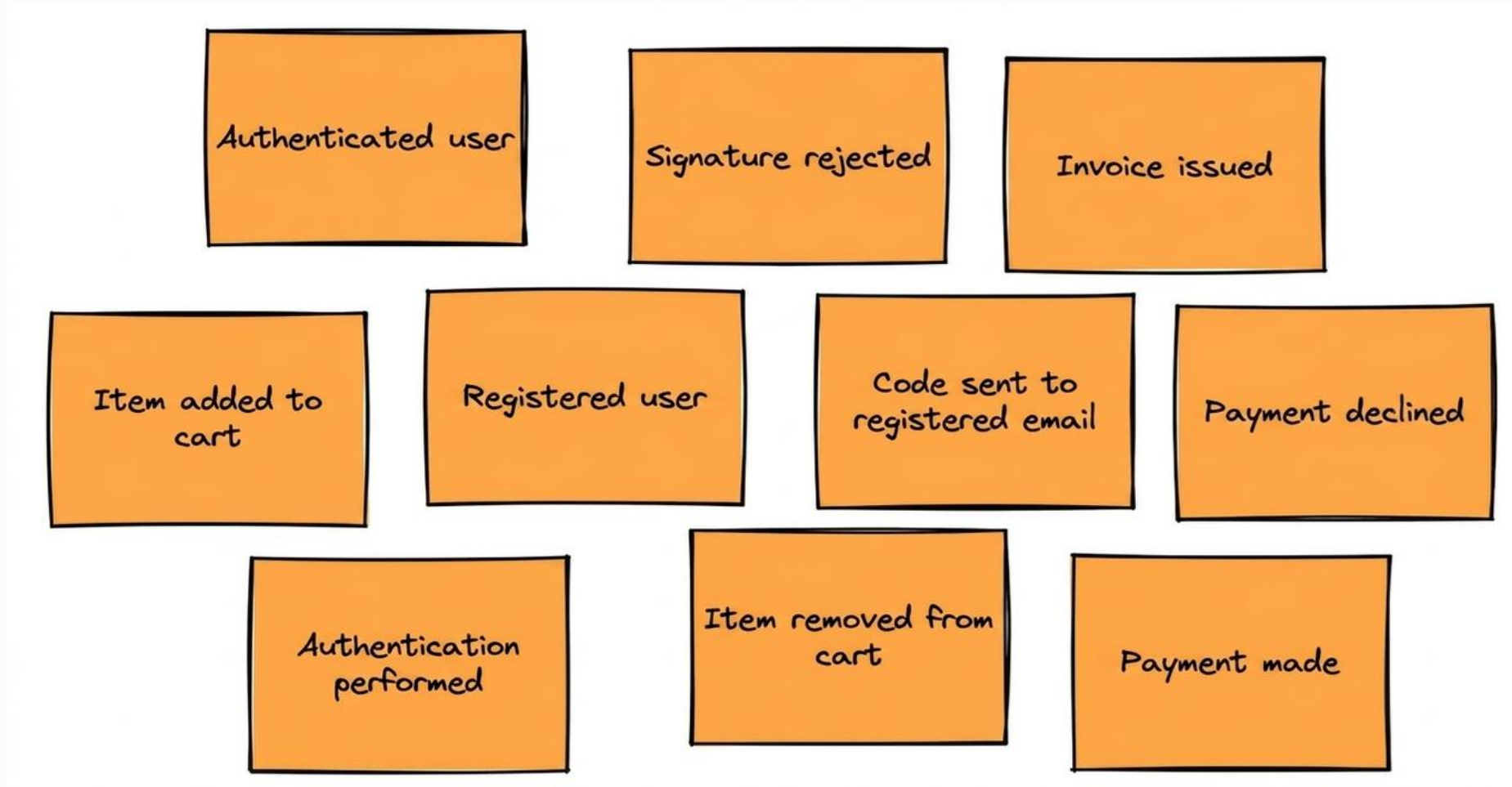


- Domain **Event**:
 - ORANGE colored paper
 - Past tense verb
 - Relevant for domain experts

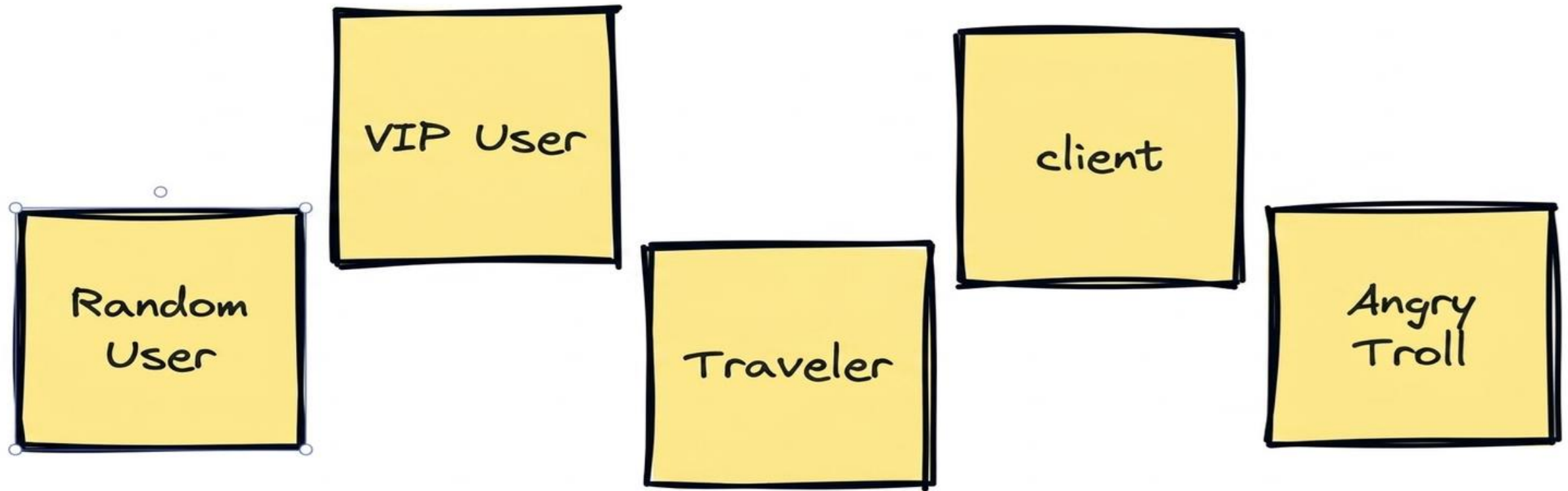
Understanding what events are



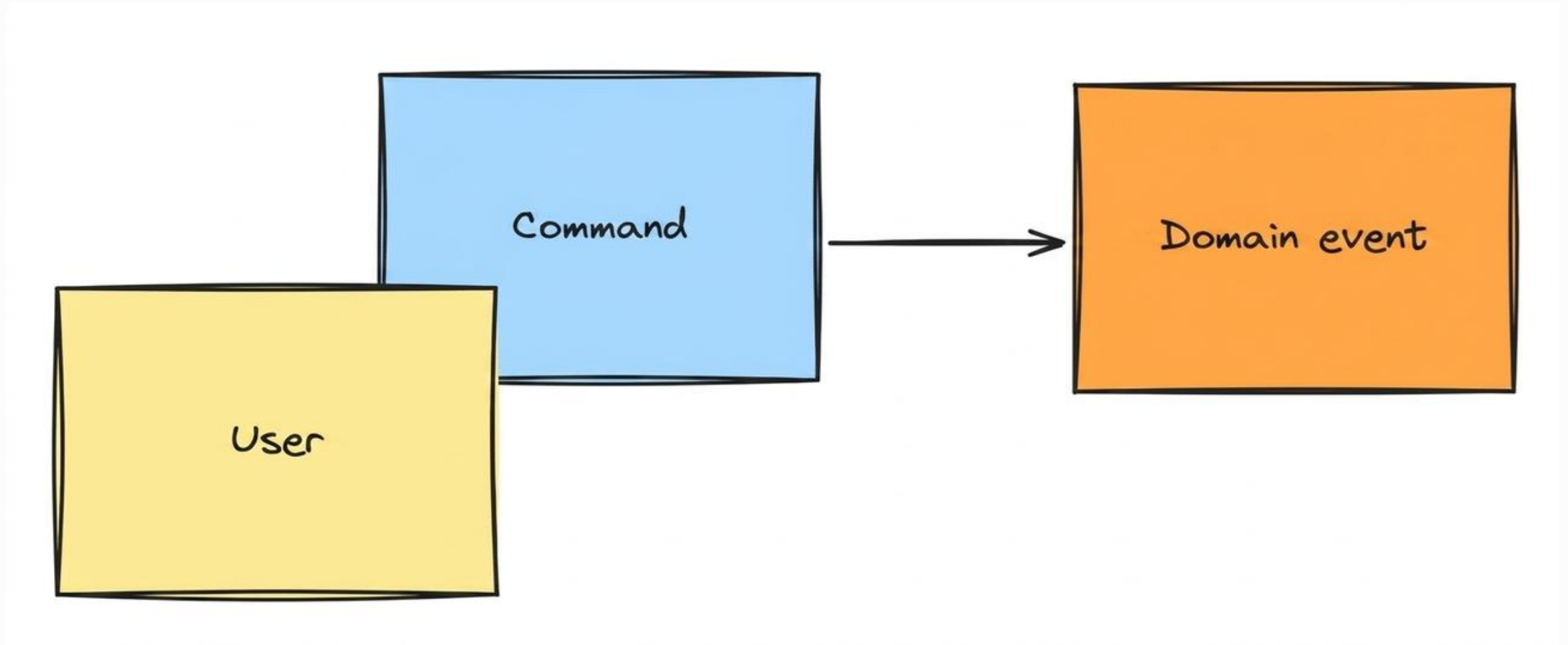
Some examples of domain events



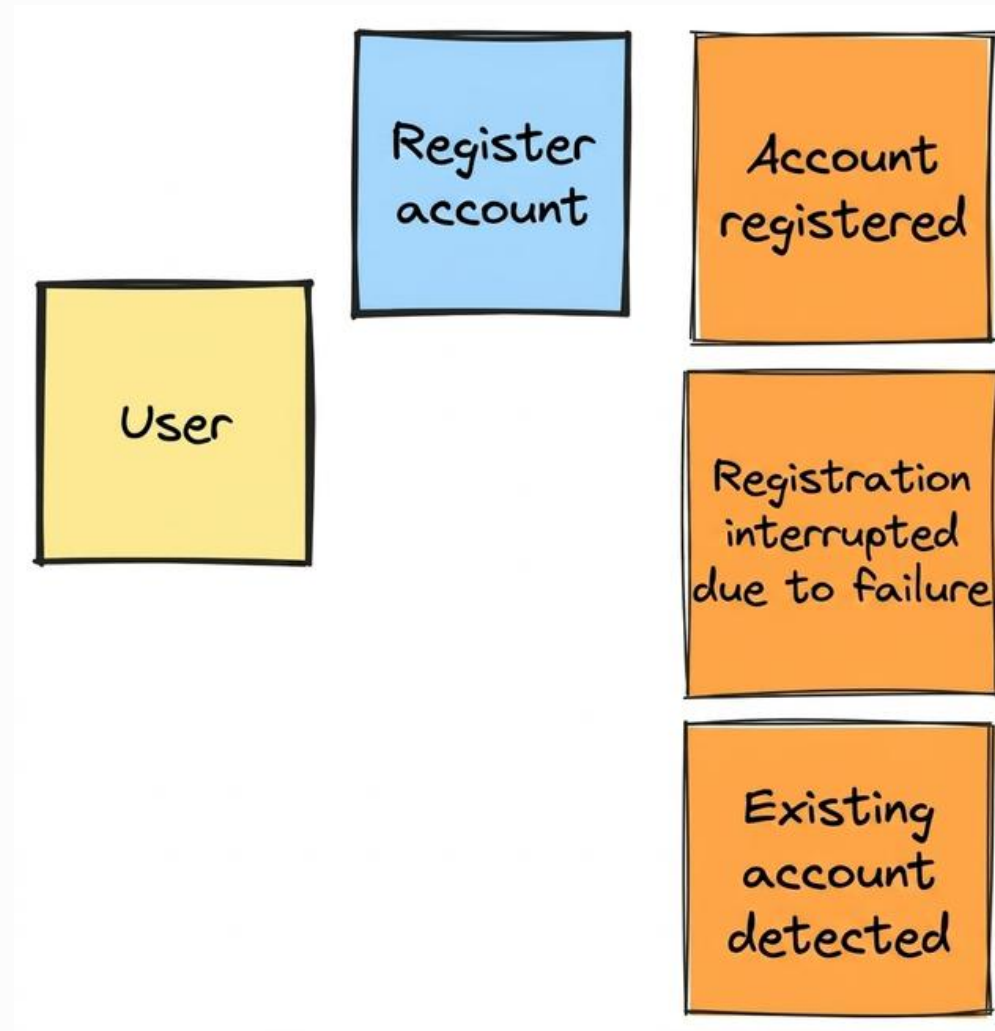
Block: Person



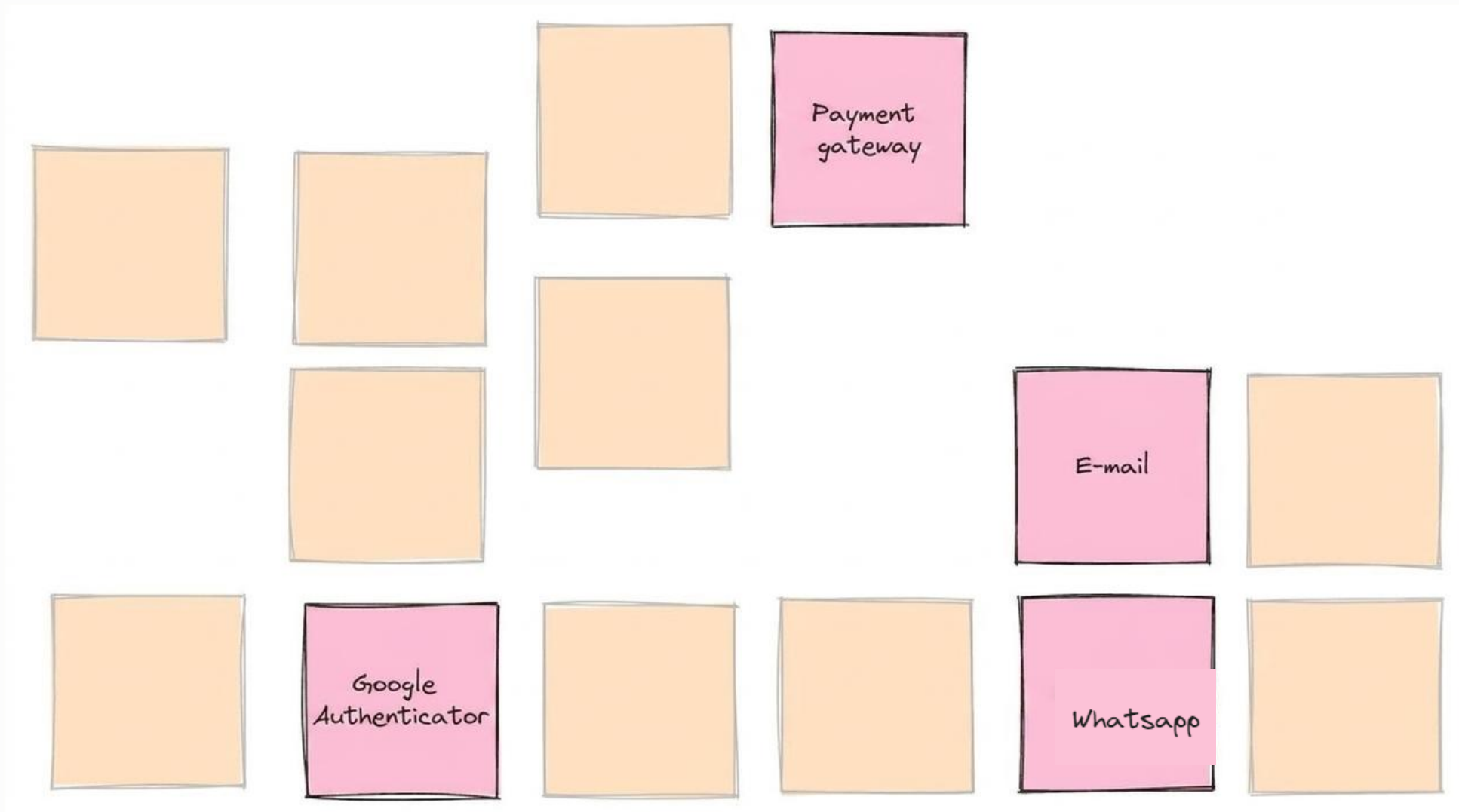
Block: Command



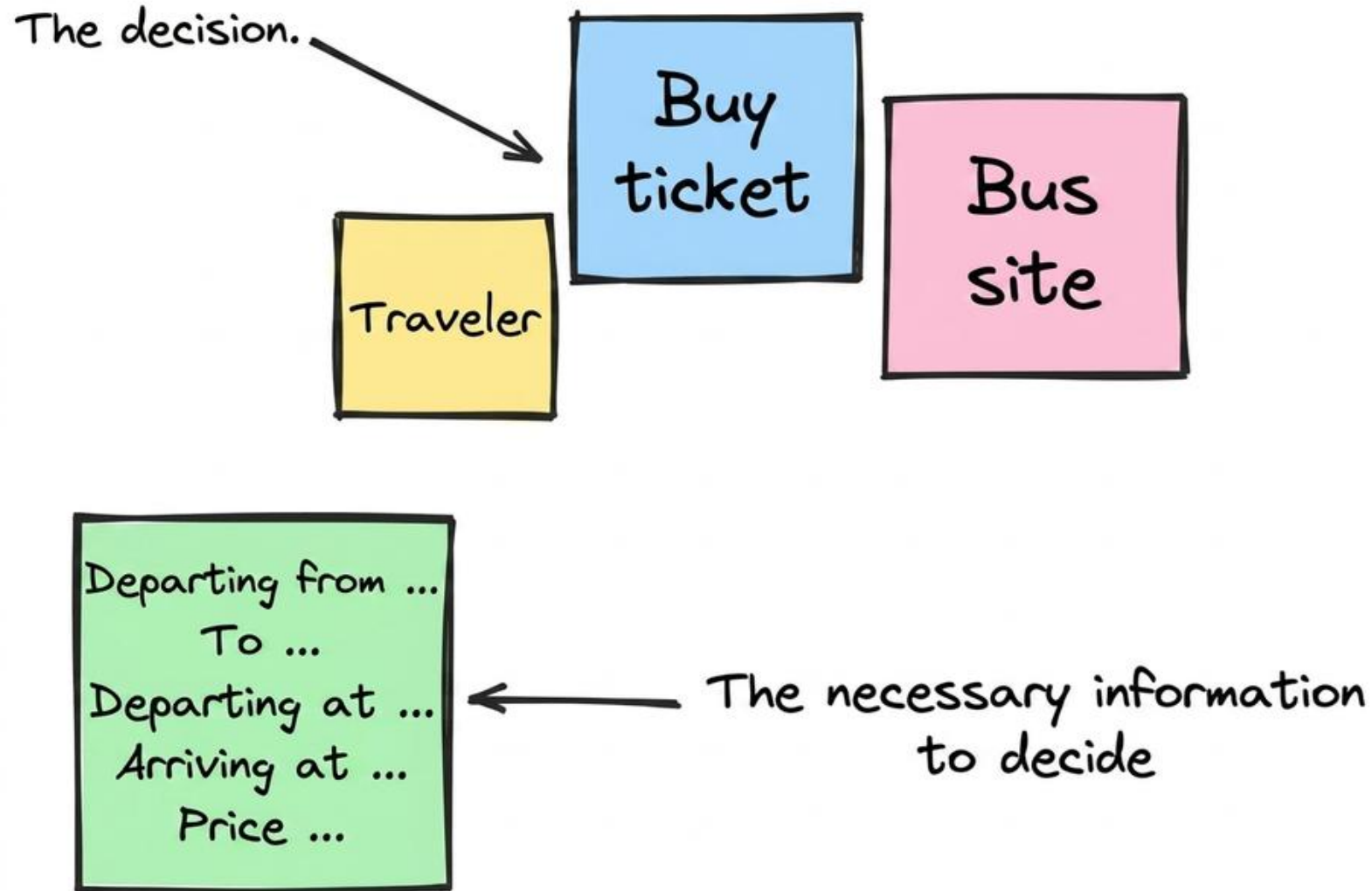
Multiple events for the same interaction



Block: External systems



Block: Query Model / Information



Block: Policy

Whenever [event(s)] then [command(s)]

Registration
completed

Welcome
policy

Send welcome
package

Example of policy for DEVs

```
# Event: "Subscription activated"
class EventoAssinaturaAtivada:
    def __init__(self, assinatura_id, usuario_email):
        self.assinatura_id = assinatura_id
        self.usuario_email = usuario_email
    }

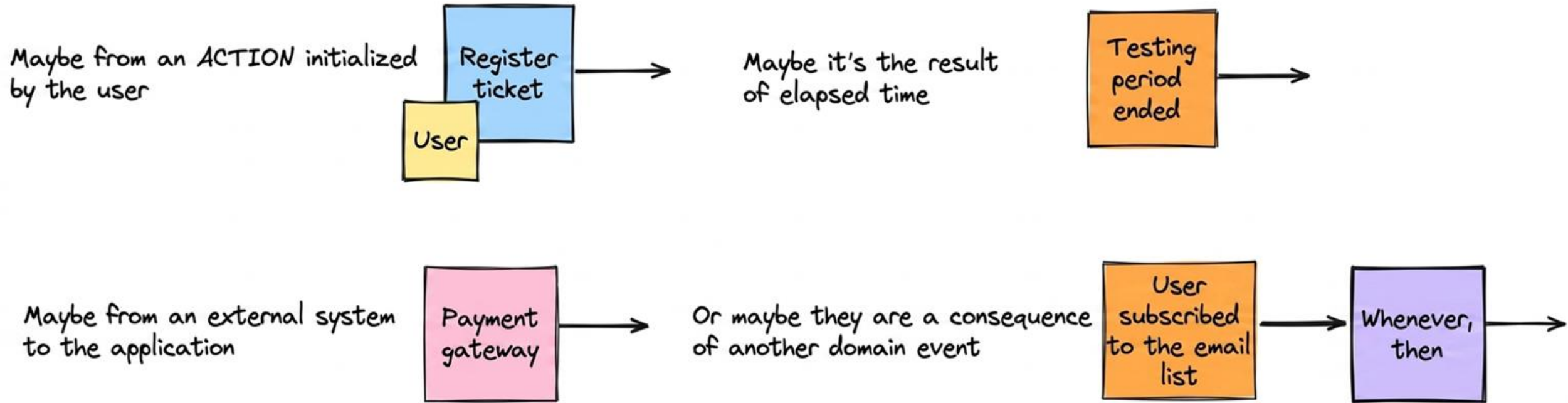
# Command: "Send welcome email"
def enviar_email_boas_vindas(email_cliente):
    # Logic to send the email
    print(f>Welcome email sent to {email_cliente}")

# Policy: "Welcome policy"
def on_evento_assinatura_ativada(evento):
    enviar_email_boas_vindas(evento.usuario_email)

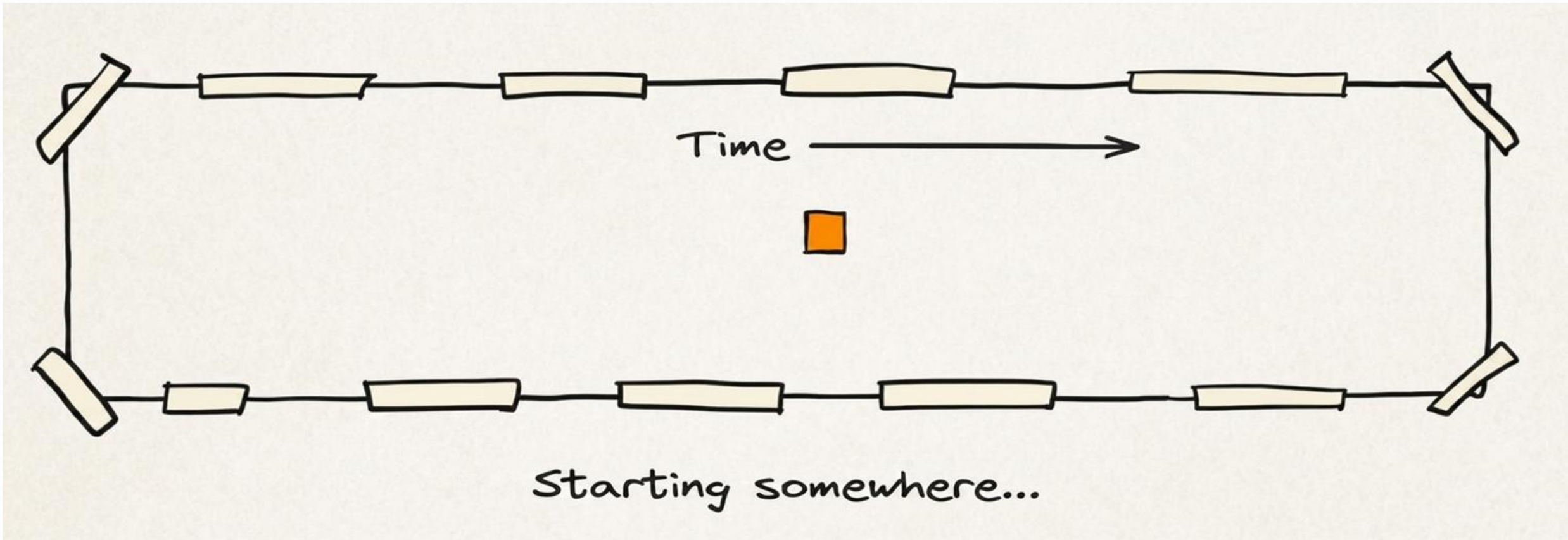
# Simulating the subscription activation event
evento_ativacao = EventoAssinaturaAtivada(assinatura_id=101, usuario_email="cliente@example.com")

# Policy reacting to the subscription activation event
on_evento_assinatura_ativada(evento_ativacao)
```

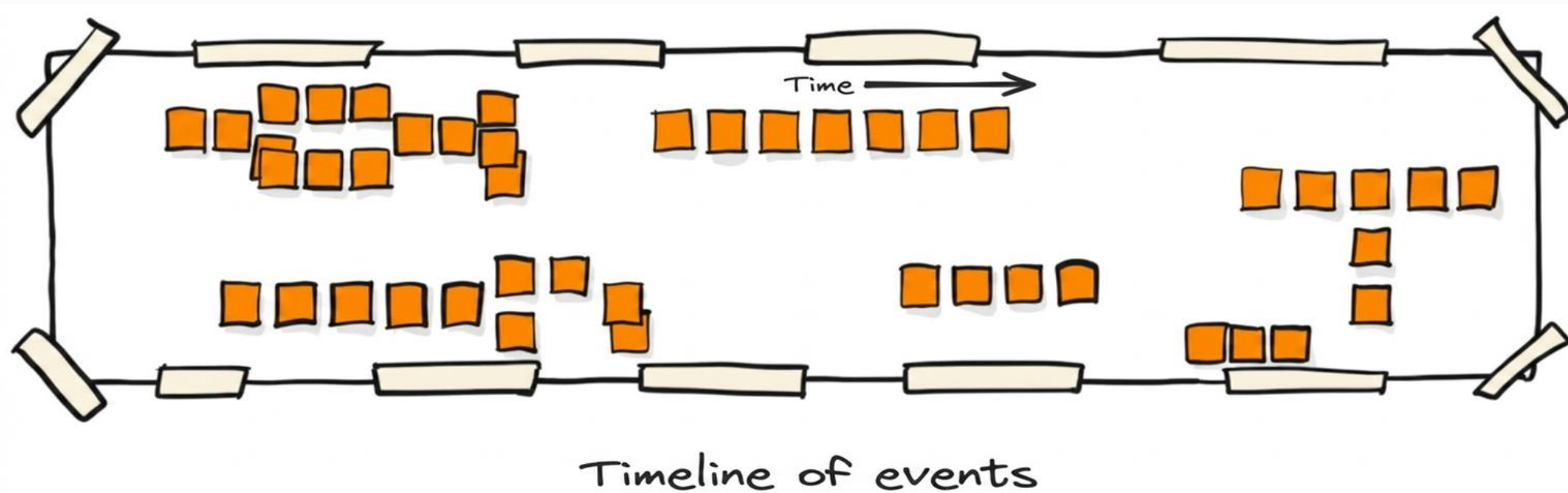
Where do domain events come from?



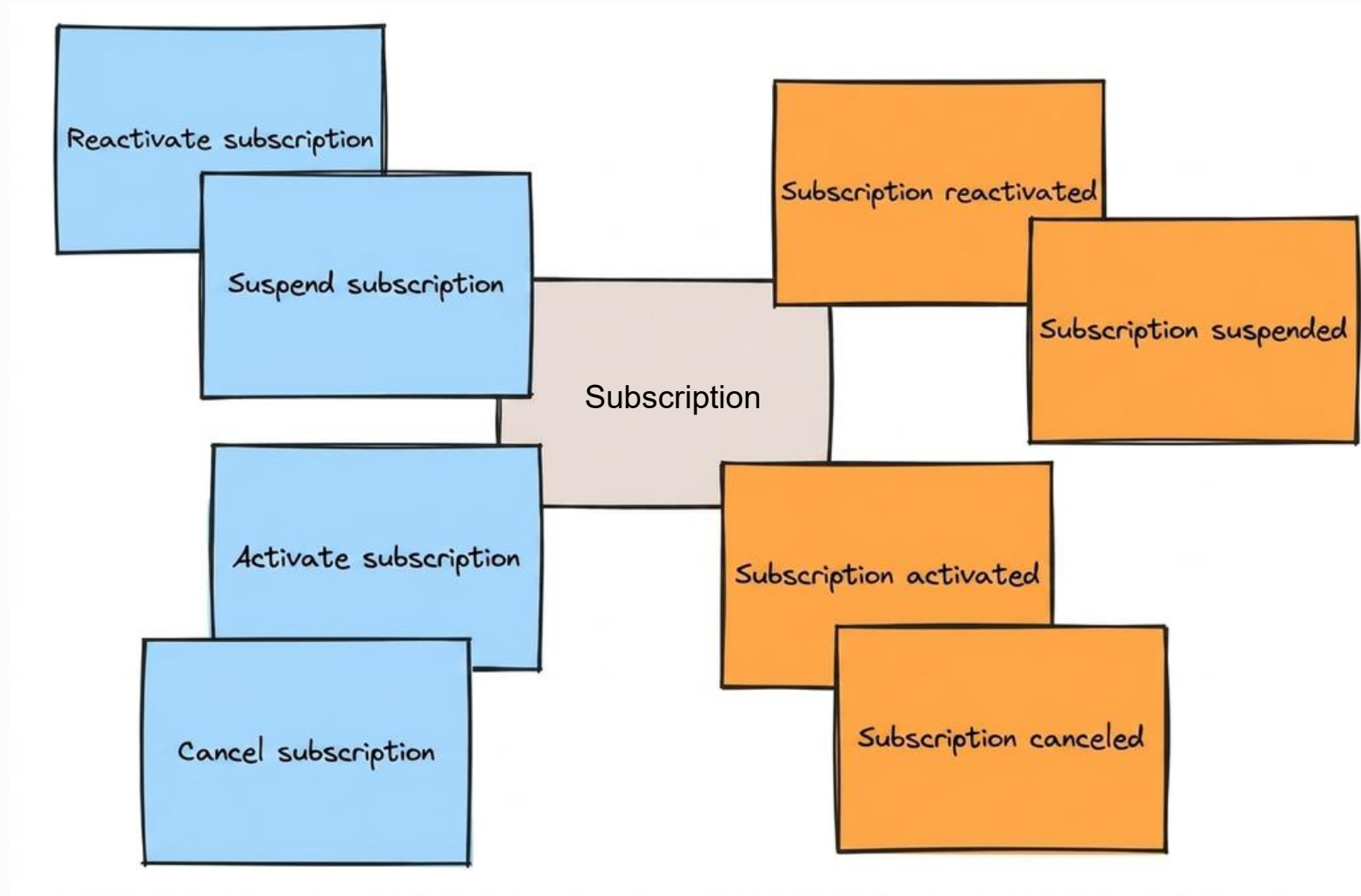
Timeline of events



Timeline of events



Block: Aggregates



Example of aggregates for DEVs

```
from datetime import datetime

class Usuario:
    def __init__(self, id, nome, email):
        self.id = id
        self.nome = nome
        self.email = email
        self.ativo = True
        self.data_criacao = datetime.now()
        self.data_atualizacao = datetime.now()

    def atualizar_nome(self, novo_nome):
        self.nome = novo_nome
        self.data_atualizacao = datetime.now()

    def atualizar_email(self, novo_email):
        self.email = novo_email
        self.data_atualizacao = datetime.now()

    def desativar(self):
        self.ativo = False
        self.data_atualizacao = datetime.now()
```

Use cases and possible events

```
usuario = Usuario(id=1, nome="João Silva", email="joao.silva@example.com")
print(f"User created: {usuario.nome}, {usuario.email}, Active: {usuario.ativo}")

# Update user name
usuario.atualizar_nome("João Pereira")
print(f"Name updated: {usuario.nome}")

# Update user email
usuario.atualizar_email("joao.pereira@example.com")
print(f"Email updated: {usuario.email}")

# Deactivate user
usuario.desativar()
print(f"User deactivated: Active: {usuario.ativo}")
```

Ex.: Subscription Aggregate

```
from datetime import datetime, timedelta

class Assinatura:
    def __init__(self, id, usuario_id, data_inicio, duracao_meses):
        self.id = id
        self.usuario_id = usuario_id
        self.data_inicio = data_inicio
        self.duracao_meses = duracao_meses
        self.data_fim = self.calcular_data_fim()
        self.ativa = True
        self.data_cancelamento = None

    def calcular_data_fim(self):
        return self.data_inicio + timedelta(days=30 * self.duracao_meses)

    def renovar(self, meses_adicionais):
        if self.ativa:
            self.duracao_meses += meses_adicionais
            self.data_fim = self.calcular_data_fim()
            print(f"Subscription renewed until {self.data_fim}")
        else:
            print("Cannot renew an inactive subscription.")

    def cancelar(self):
        self.ativa = False
        self.data_cancelamento = datetime.now()
        print(f"Subscription canceled on {self.data_cancelamento}")

    def ativar(self):
        if not self.ativa:
            self.ativa = True
            self.data_fim = self.calcular_data_fim()
            self.data_cancelamento = None
            print(f"Subscription reactivated until {self.data_fim}")
        else:
            print("The subscription is already active.")
```

Ex.: Subscription Aggregate

```
# Create a new subscription
data_inicio = datetime.now()
assinatura = Assinatura(id=1, usuario_id=123, data_inicio=data_inicio, duracao_meses=6)
print(f"Assinatura criada: {assinatura.data_inicio} até {assinatura.data_fim},
      Ativa: {assinatura.ativa}")

# Renew subscription for 3 more months
assinatura.renovar(3)

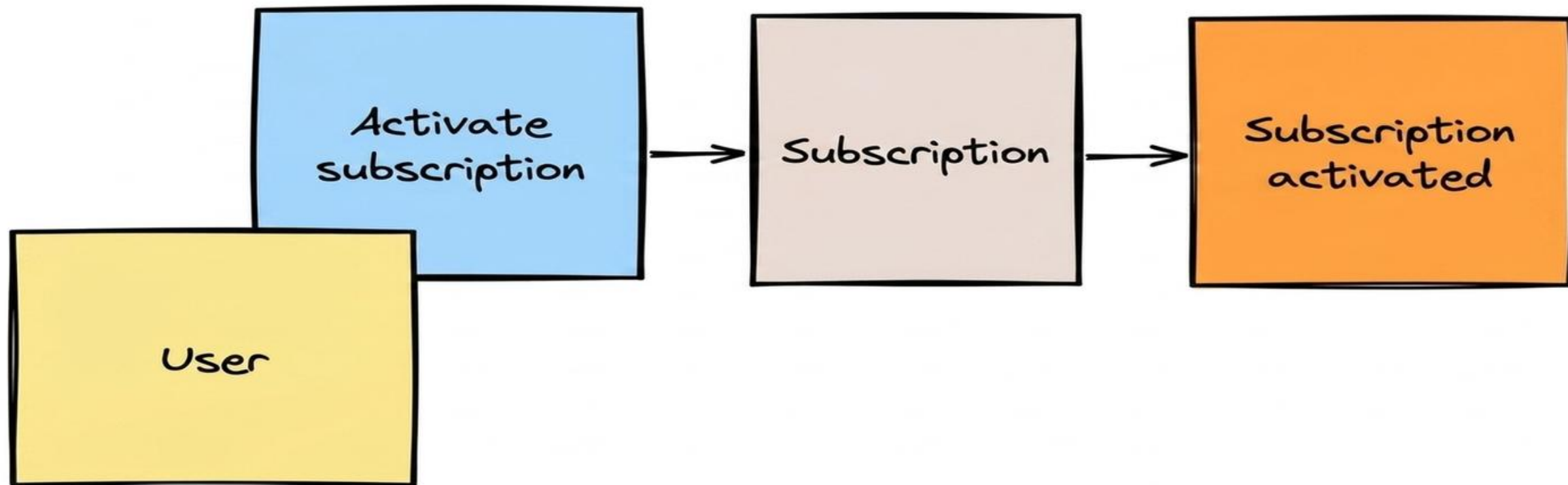
# Cancel subscription
assinatura.cancelar()

# Try to renew canceled subscription
assinatura.renovar(2)

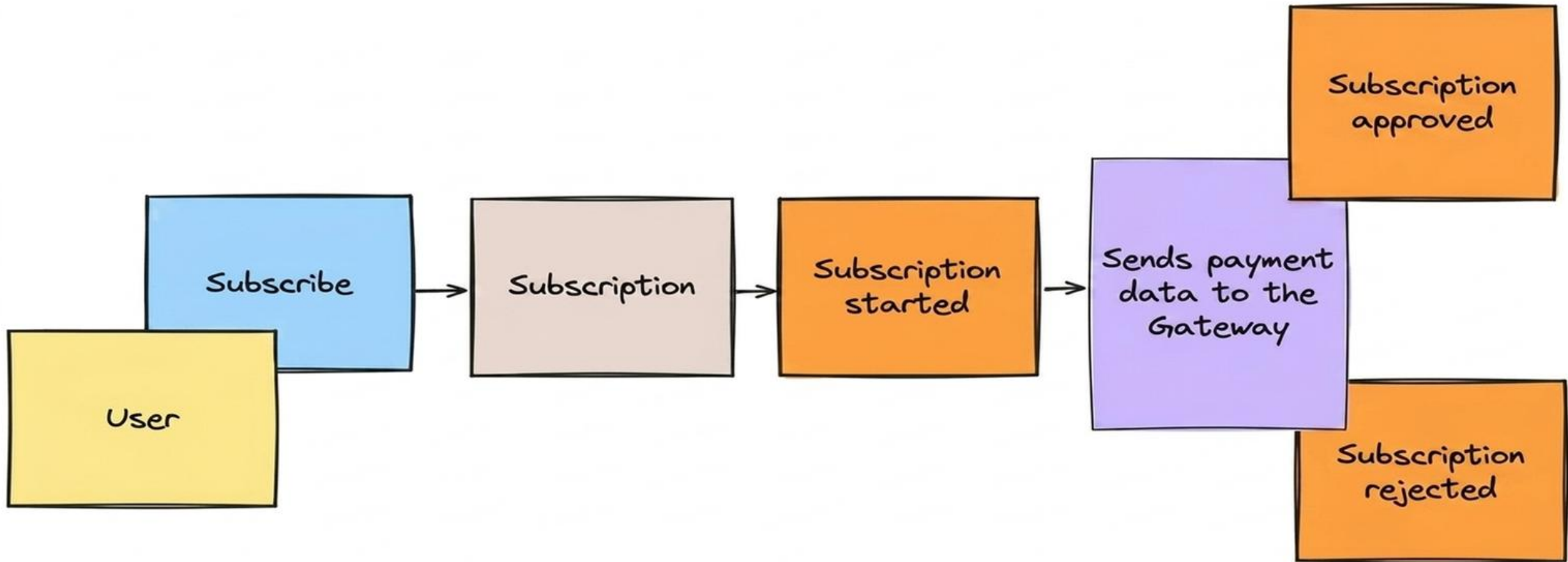
# Activate subscription again
assinatura.ativar()

# Renew reactivated subscription
assinatura.renovar(2)
```

Example view in Event Storming

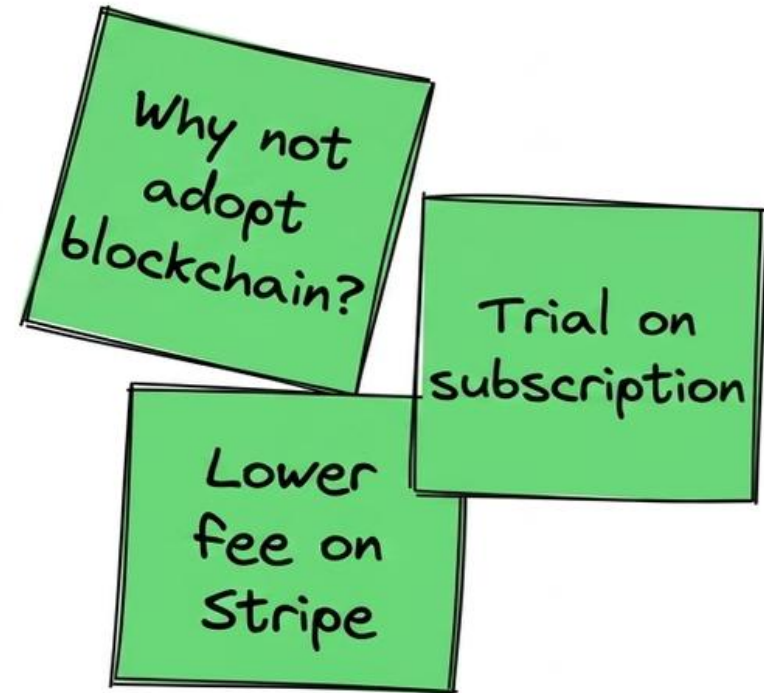
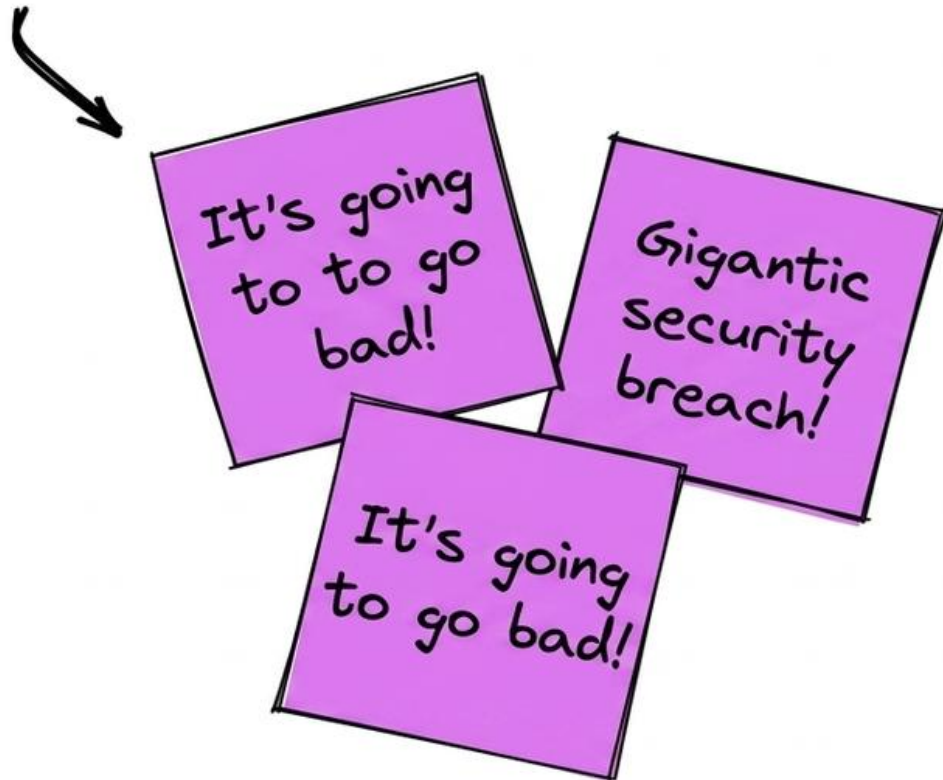


... something closer to reality



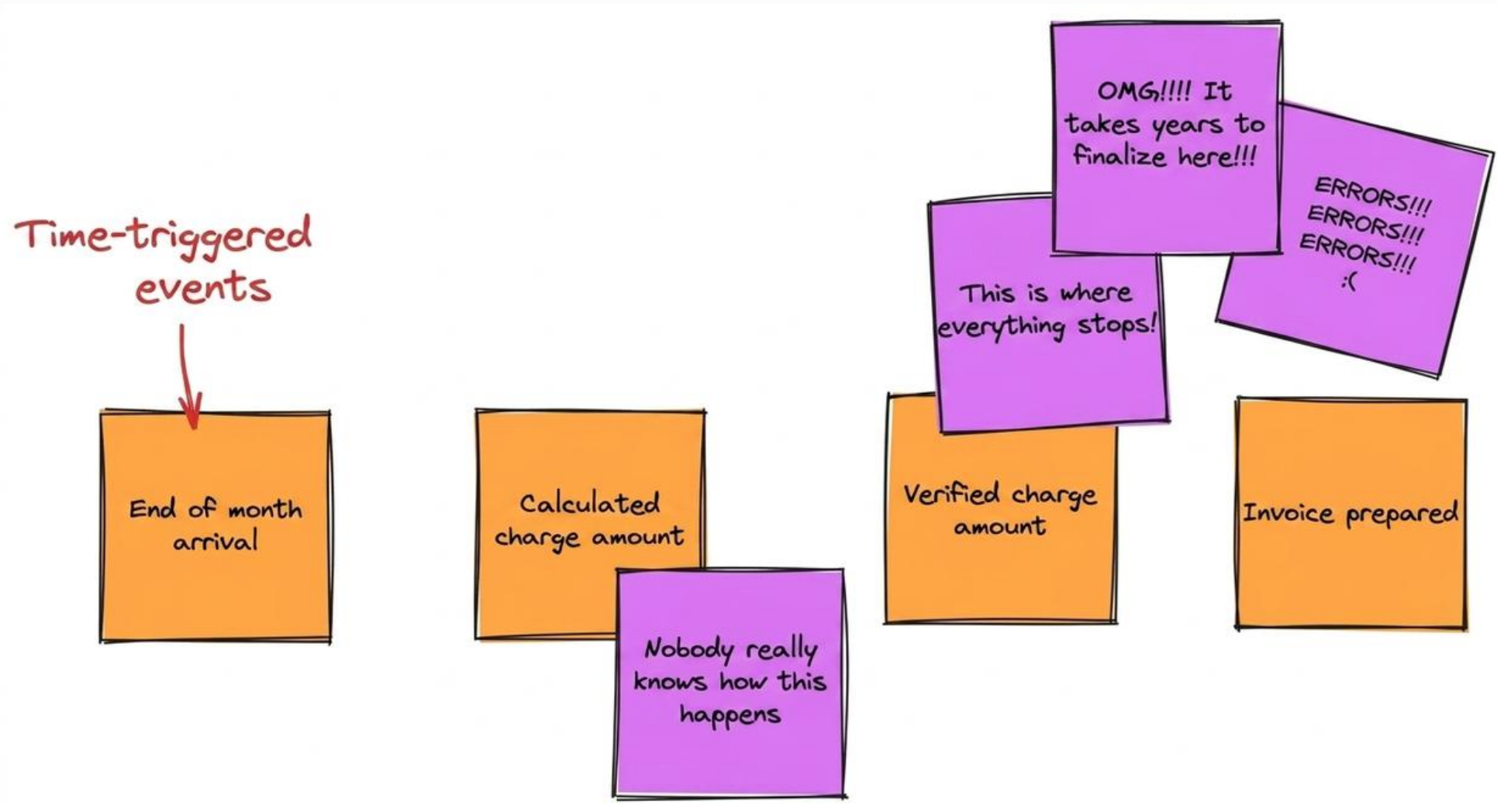
Blocks: Hotspots and Opportunities

Hotspots: are used to map problems, risks, etc.

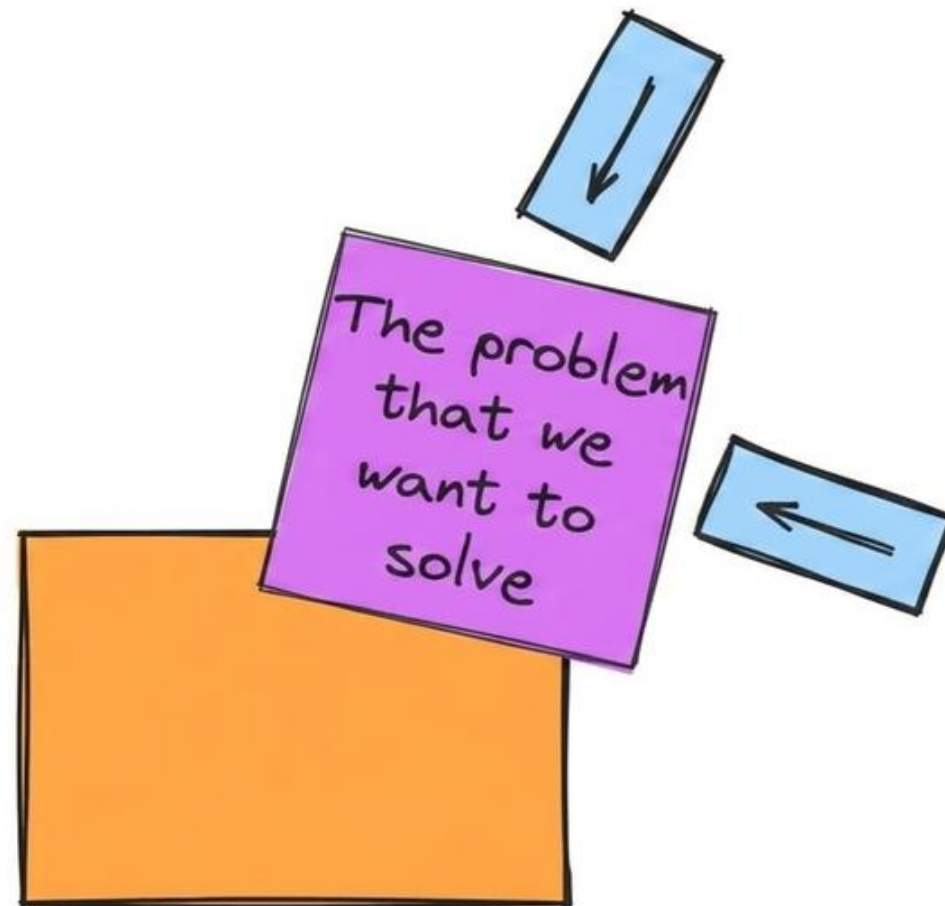
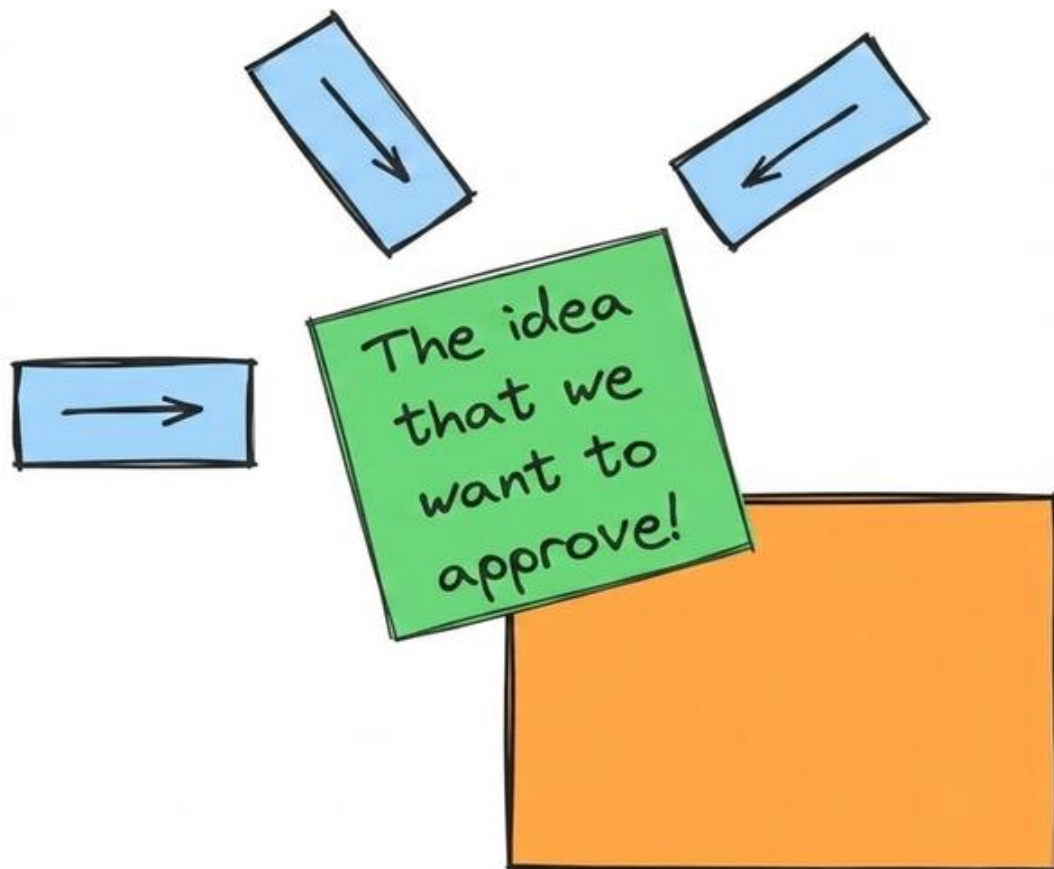


Opportunity: are used to capture ideas.

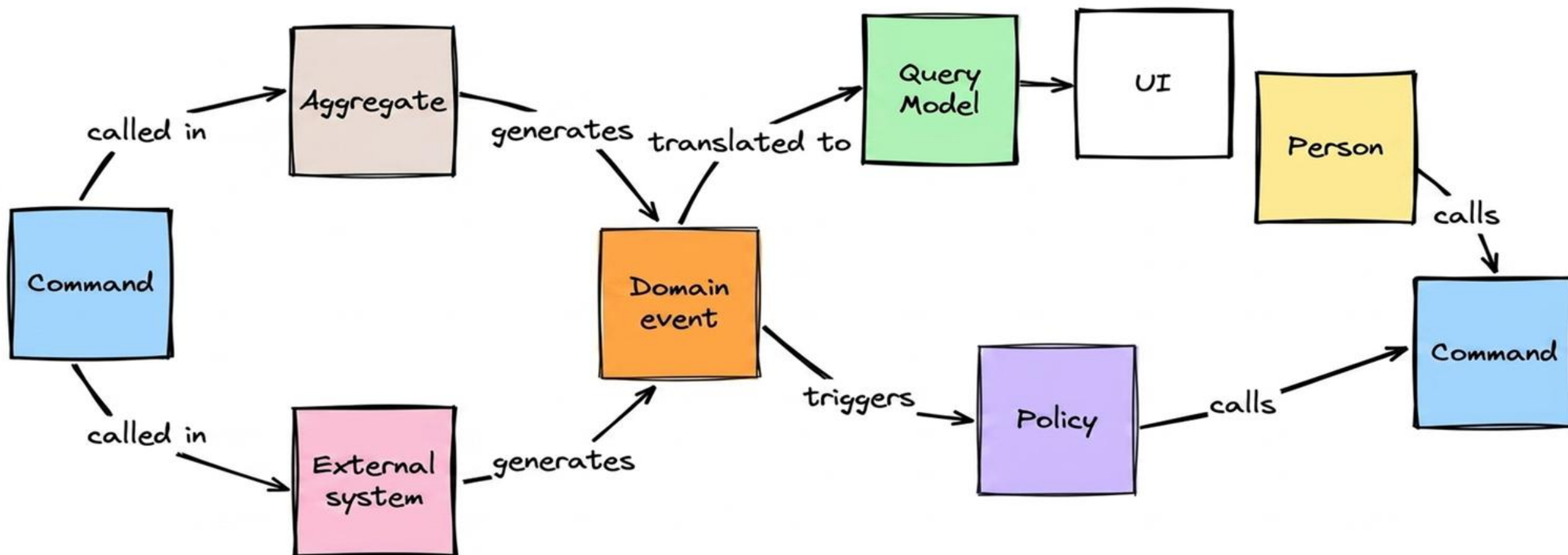
Using Hotspots



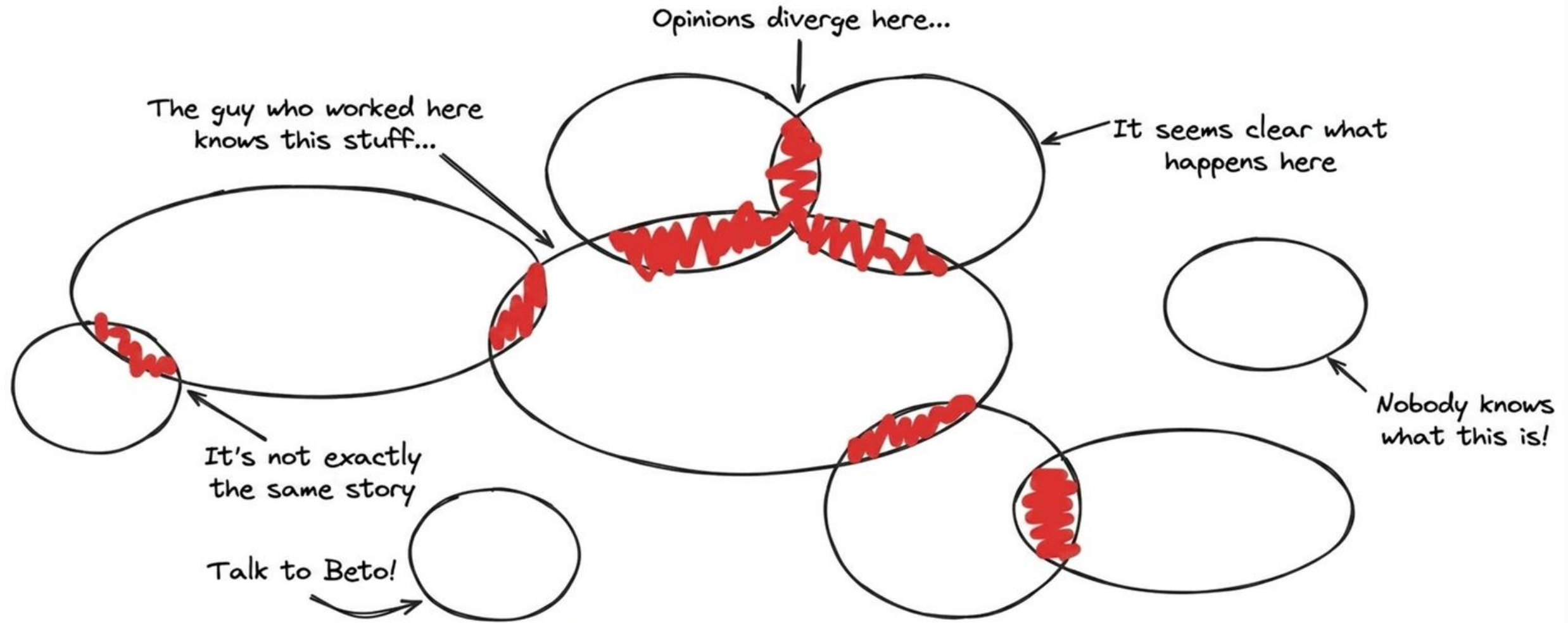
Block: Arrow Voting



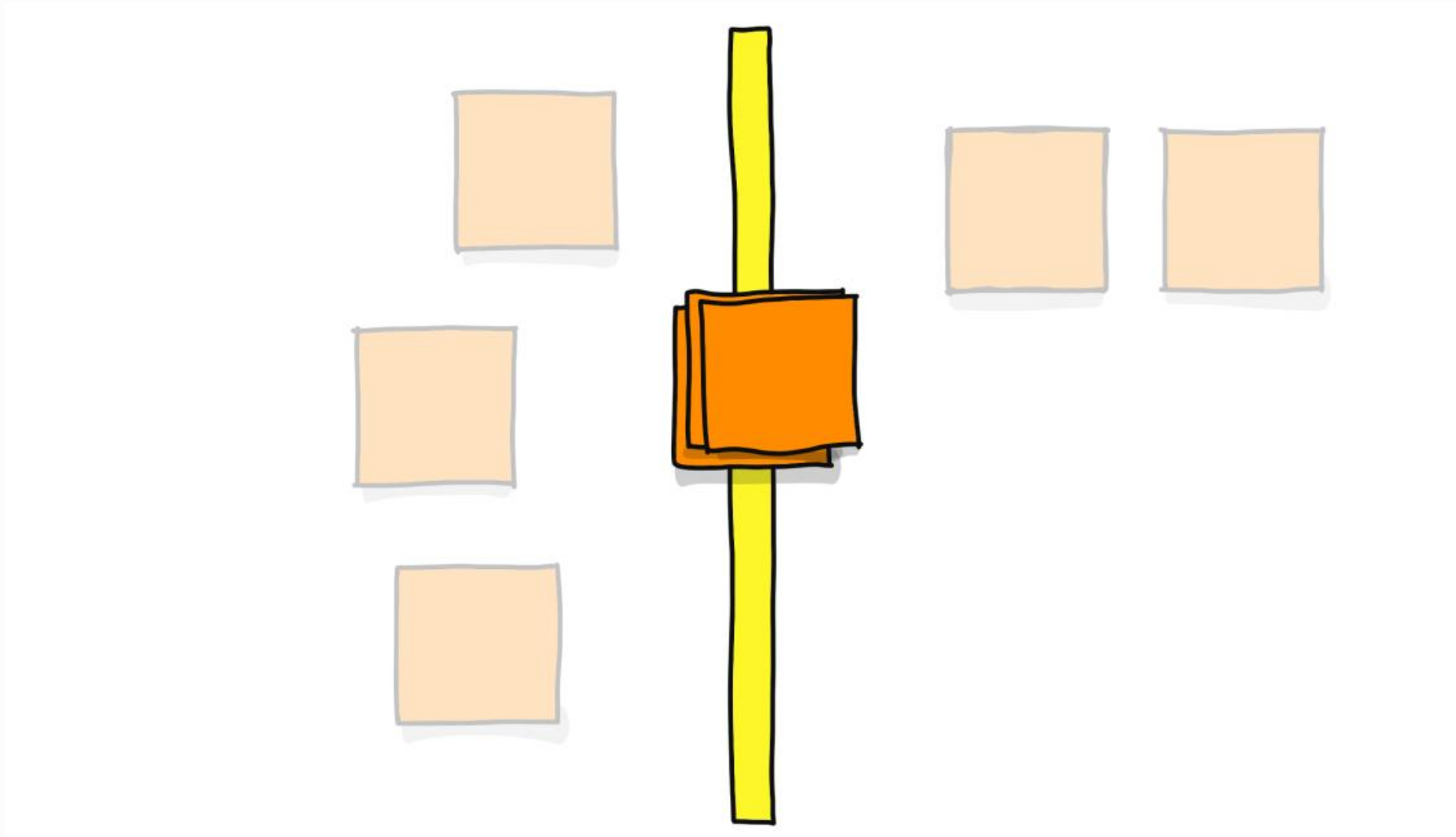
"The figure that explains everything"



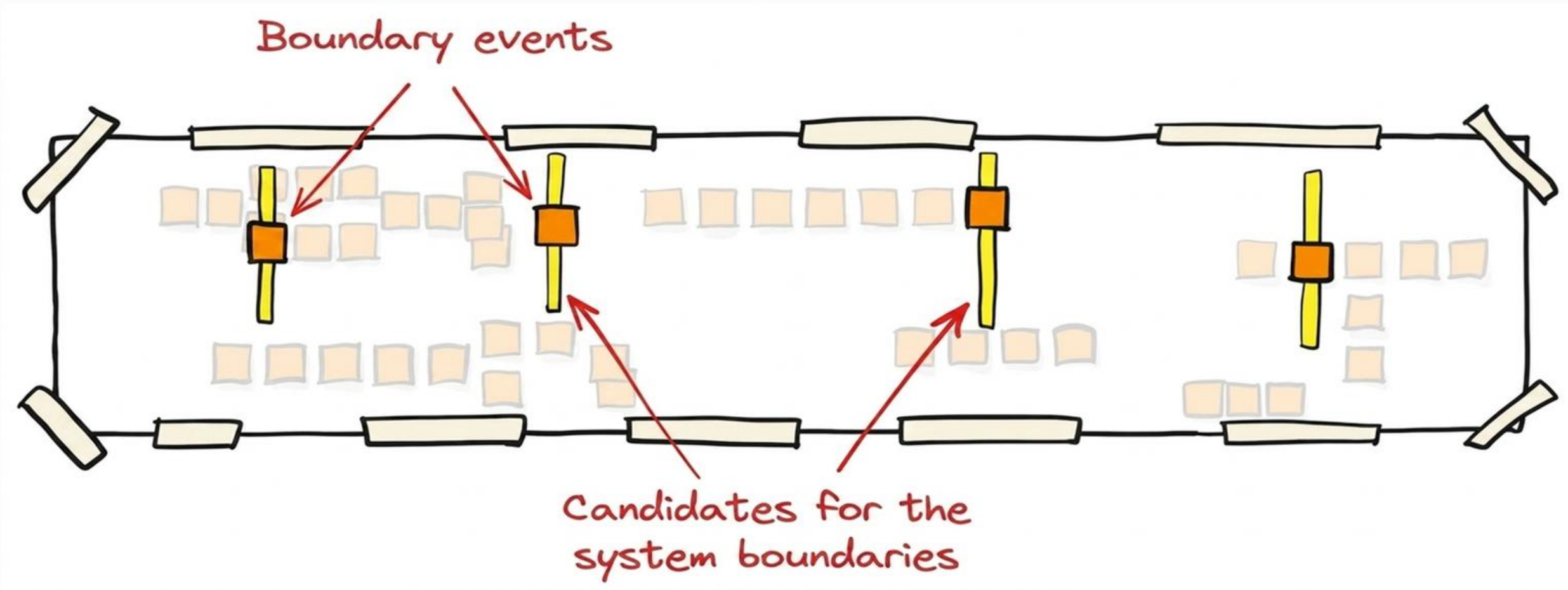
Bounded Contexts



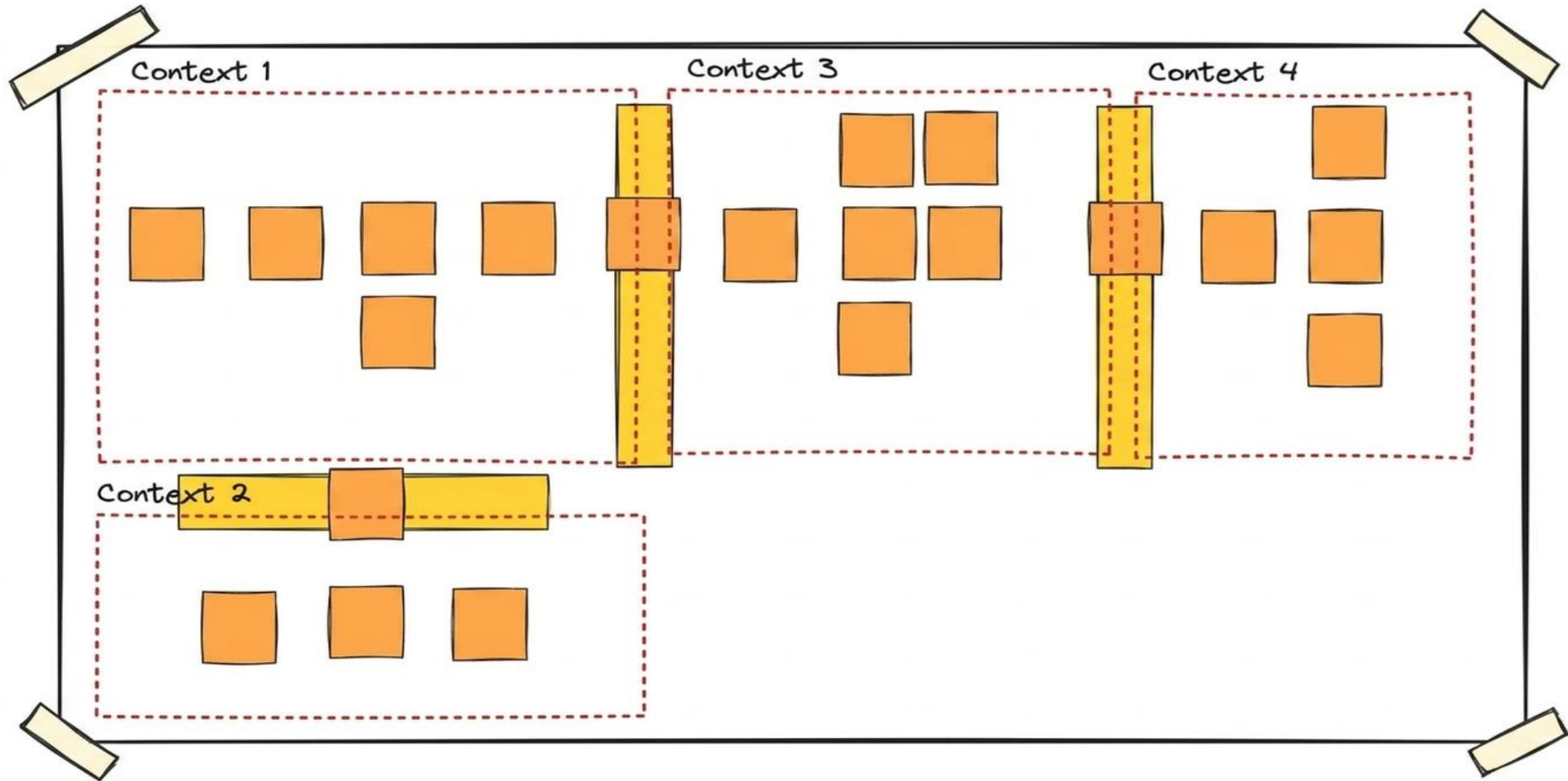
Boundary events



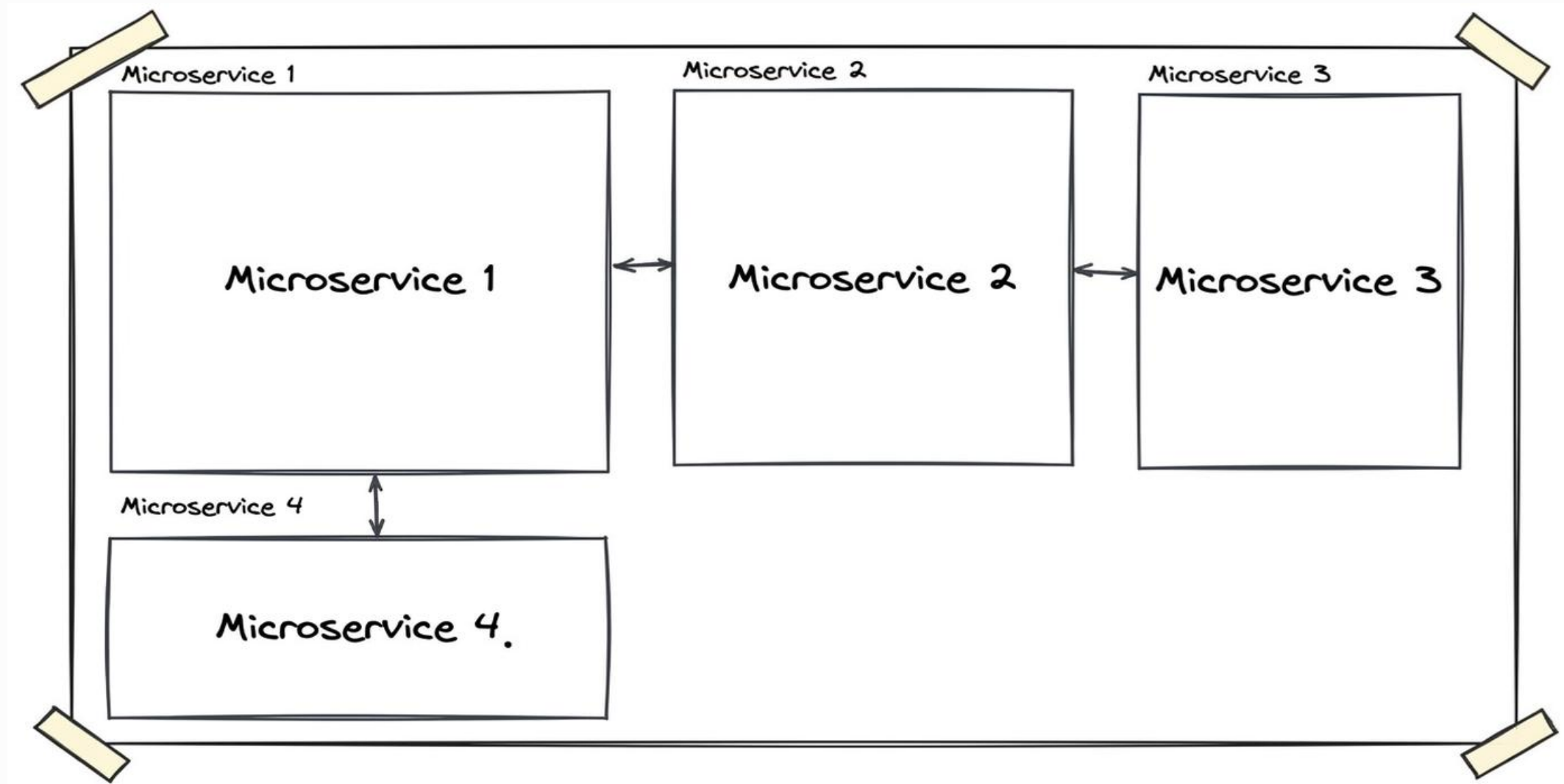
Discovering Bounded Contexts



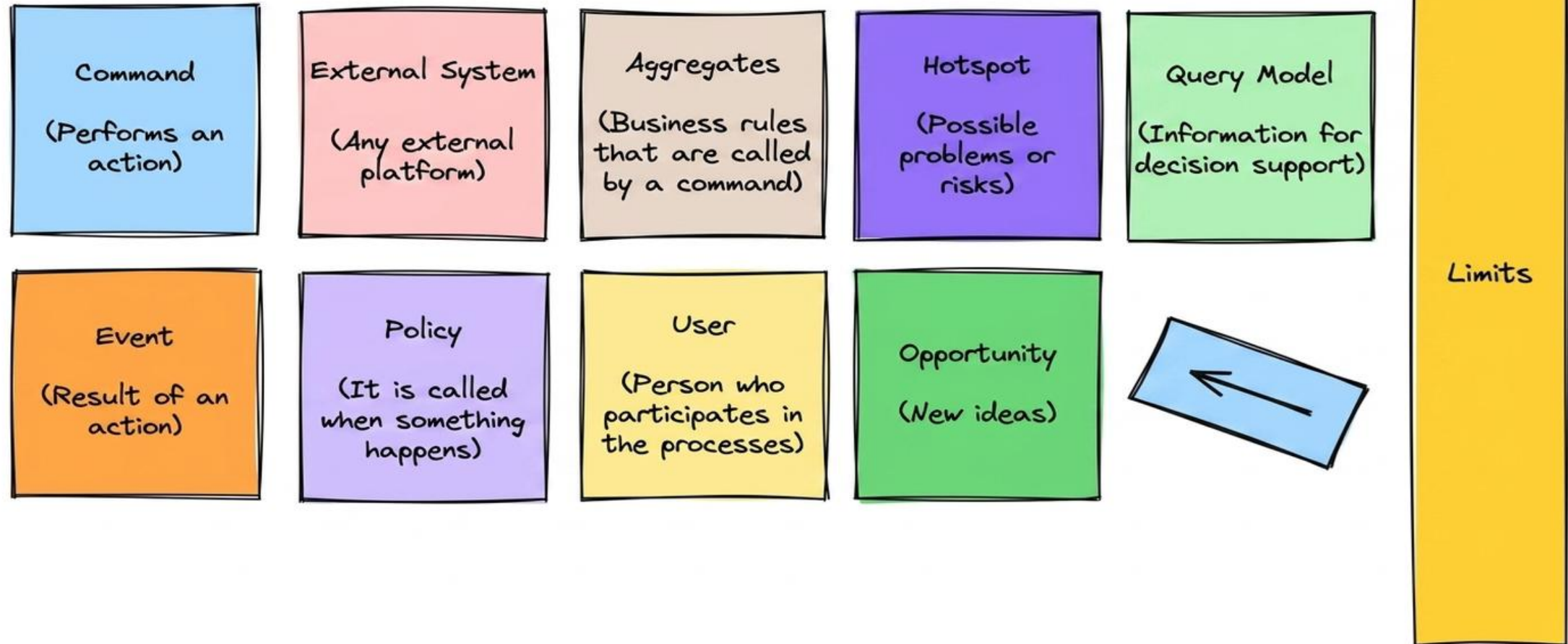
Visualizing the Contexts



Visualizing possible microservices



Reviewing the Event Storming blocks







Share

Library

Export, preferences, languages...

To move the screen, hold the mouse wheel or the spacebar while dragging or use the hand tool

Choose a tool and start drawing!



All data is saved locally in your browser.

- Open Ctrl+O
- Help ?
- Live collaboration...
- Sign up

Shortcuts and help

100%

<https://excalidraw.com/>





Suggested Readings/References

1. BRANDOLINI, A. Introducing EventStorming. [s.l.] Leanpub, 2015.
2. VLAD KHONONOV. Learning Domain-Driven Design. [s.l.] “O’Reilly Media, Inc.”, 2021.

Thank you!

Helder Prado Santos | [linkedin.com/in/helderprado](https://www.linkedin.com/in/helderprado)

